



**UNIVERSIDADE FEDERAL DO TOCANTINS
CÂMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

HENRIQUE NORONHA FERNANDES

**ANÁLISE COMPARATIVA DE GERENCIAMENTO DE RECURSOS DO
SO: ABLETON LIVE VS. FL STUDIO**

PALMAS (TO)

2026

Henrique Noronha Fernandes

**Análise Comparativa de Gerenciamento de Recursos do SO: Ableton Live vs.
FL Studio**

Relatório de Sistemas Operacionais
apresentado à Universidade Federal do
Tocantins para obtenção do título de Bacharel
em Ciência da Computação. Orientador:

[Titulação] [Nome do Professor de SO]
[Sobrenome]

PALMAS (TO)

2026

SUMÁRIO

1	INTRODUÇÃO	0
2	REFERENCIAL TEÓRICO	1
2.1	Gerenciamento de Processos e <i>Threads</i>	1
2.1.1	Escalonador do Windows NT	1
2.1.2	Tempo Real Crítico vs. Não Crítico	1
2.2	Interrupções e Latência de I/O em Tempo Real	1
2.2.1	<i>Callback</i> de Áudio e Tamanho de <i>Buffer</i>	2
2.2.2	ASIO vs. WDM/DirectSound	2
2.2.3	Por que DAWs não usam Escalonamento de Prazos	2
2.3	Gerenciamento de Memória e Paginação	2
2.3.1	<i>Eager</i> vs. <i>Lazy Allocation</i>	3
2.3.2	<i>Private Bytes</i>	3
3	METODOLOGIA	4
3.1	Ambiente de Teste e Isolamento de Variáveis	4
3.2	Instrumentação e Coleta de Métricas	4
3.3	Protocolo de Teste 1: Consumo em Repouso (<i>Idle</i>)	5
3.4	Protocolo de Teste 2: Carga <i>I/O-bound</i> com 20 Trilhas AIFF .	5
3.5	Protocolo de Teste 3: Escalonamento de <i>Threads</i>	5
3.6	Protocolo de Teste 4: Latência com Driver Nativo (LatencyMon)	5
3.7	Protocolo de Teste 5: Estresse CPU-bound e Ponto de Ruptura	5
4	RESULTADOS	7
4.1	Teste 1 — Consumo em Repouso (<i>Idle</i>)	7
4.1.1	CPU em repouso	7

4.1.2	RAM em repouso	8
4.1.3	<i>Threads</i> em repouso	8
4.2	Teste 2 — Carga <i>I/O-bound</i>: 20 Trilhas AIFF	8
4.2.1	CPU com 20 trilhas	9
4.2.2	RAM com 20 trilhas	9
4.2.3	<i>Threads</i> com 20 trilhas	10
4.3	Teste 3 — Escalonamento de <i>Threads</i> (Process Explorer)	11
4.3.1	Prioridade de escalonamento	11
4.3.2	Composição de <i>threads</i> por módulo	11
4.4	Teste 4 — Latência com Driver Nativo (LatencyMon)	12
4.4.1	Latência <i>interrupt-to-process</i>	12
4.4.2	<i>Hard pagefaults</i> e tempo de CPU em DPCs/ISRs	13
4.5	Teste 5 — Estresse CPU-bound e Ponto de Ruptura	14
4.5.1	Ponto de ruptura por driver	14
4.6	Consumo de CPU e Comportamento Multithread	15
4.7	Análise de Latência e Buffer	15
5	DISCUSSÃO	16
5.1	Análise Comparativa — Teste 1 (Repouso)	16
5.2	Análise Comparativa — Teste 2 (20 Trilhas AIFF)	16
5.3	Análise Comparativa — Teste 3 (Escalonamento de <i>Threads</i>) .	16
5.4	Análise Comparativa — Teste 4 (Latência)	16
5.5	Análise Comparativa — Teste 5 (Estresse CPU-bound)	17
6	CONCLUSÃO	18
	REFERÊNCIAS	19

1 INTRODUÇÃO

Historicamente, os Sistemas Operacionais (SO) evoluíram de simples monitores de execução sequencial para ecossistemas complexos de gerenciamento de hardware e software. Enquanto as primeiras gerações de sistemas focavam estritamente na abstração de comandos básicos e organização de arquivos, os sistemas contemporâneos precisam orquestrar uma arquitetura de múltiplos núcleos (*multicore*), memória virtual e interrupções em milissegundos. No cenário contemporâneo, os computadores pessoais atuam como Estações de Trabalho de Áudio Digital (DAWs — *Digital Audio Workstations*), executando softwares complexos que exigem do SO uma orquestração extremamente eficiente dos componentes de hardware. Nesse contexto, a performance de uma DAW não depende apenas da força bruta do processador, mas de como o sistema operacional gerencia as *threads* de execução, aloca a memória e lida com operações de entrada e saída (I/O) em alta velocidade.

O grande desafio imposto por essas aplicações é a necessidade de operar sob restrições de tempo real. Conforme a classificação de Silberschatz, Galvin e Gagne (2015), as DAWs enquadram-se como sistemas de tempo real não crítico (*soft real-time*). Nesses sistemas, embora o atraso no atendimento de um processo não cause uma falha fatal no computador, ele resulta em uma severa degradação da qualidade do serviço — nesse caso, manifestando-se como falhas e artefatos sonoros indesejados. Para evitar essa degradação, o sistema operacional precisa garantir que as *threads* de áudio tenham preferência sobre processos comuns, minimizando a latência de despacho e a latência de interrupção.

Diante dessa problemática, o presente trabalho realiza uma análise comparativa do consumo e gerenciamento de recursos do sistema operacional entre duas das principais DAWs do mercado: Ableton Live e FL Studio. Busca-se mapear como cada arquitetura de software impacta o consumo de CPU, o comportamento *multithread*, o uso de memória RAM e a latência de I/O sob as mesmas condições de estresse, demonstrando na prática os impactos das decisões de escalonamento e arquitetura do SO em ambientes de alta exigência em tempo real.

2 REFERENCIAL TEÓRICO

2.1 Gerenciamento de Processos e *Threads*

Um **processo** é um programa em execução, dotado de espaço de endereçamento próprio e dos recursos a ele atribuídos pelo SO (SILBERSCHATZ; GALVIN; GAGNE, 2015). Dentro de um processo podem coexistir múltiplas ***threads***: unidades menores de execução que compartilham o mesmo espaço de endereçamento, mas mantêm pilha e registradores próprios (TANENBAUM; BOS, 2016). Essa partilha torna a troca de contexto entre *threads* muito mais barata do que entre processos — razão pela qual motores de áudio recorrem ao *multithreading* para distribuir o processamento de blocos de amostras entre os núcleos do hardware.

2.1.1 Escalonador do Windows NT

O escalonador do Windows NT é **preemptivo baseado em prioridades**, com 32 níveis (0–31): a faixa de usuário ocupa os níveis 1–15 e a faixa de tempo real, 16–31 (STALLINGS, 2017). Os valores observados neste trabalho situam-se na faixa de usuário: *Normal* (8), *Above Normal* (10) e *High* (12–13). A cada *thread* o sistema atribui uma **prioridade base** (derivada da classe do processo) e, em tempo de execução, pode aplicar um ***priority boosting*** temporário — elevação automática para compensar *starvation* ou premiar *threads* recém-acordadas de I/O (STALLINGS, 2017). O *boost* decai gradualmente até retornar à base, o que explica a divergência entre prioridade base e dinâmica observável em ferramentas como o *Process Explorer*.

2.1.2 Tempo Real Crítico vs. Não Crítico

Silberschatz, Galvin e Gagne (2015) distinguem dois regimes de tempo real. Em sistemas de **tempo real crítico** (*hard real-time*), perder um prazo é uma falha absoluta — inaceitável em aviãoica ou em dispositivos médicos. Em sistemas de **tempo real não crítico** (*soft real-time*), a perda eventual de um prazo degrada a qualidade do serviço, mas não compromete o sistema. DAWs caem nesta segunda categoria: um *buffer* não preenchido a tempo produz *dropout* audível, mas a máquina continua estável. Basta, portanto, que as *threads* de áudio operem em prioridade superior à dos processos comuns para que o escalonador as despache primeiro sob contenção.

2.2 Interrupções e Latência de I/O em Tempo Real

Uma **interrupção de hardware** é um sinal assíncrono pelo qual um periférico avisa o processador de que requer atenção imediata (SILBERSCHATZ; GALVIN;

GAGNE, 2015). Ao recebê-lo, o processador suspende a *thread* corrente, salva seu estado e transfere o controle para a **Rotina de Serviço de Interrupção** (ISR). O intervalo entre o sinal e o início da ISR é a **latência de interrupção**; somada ao tempo de troca de contexto até a *thread* destino, resulta na **latência de despacho** (STALLINGS, 2017). Em áudio, qualquer atraso de despacho que exceda o período do *buffer* se manifesta como silêncio ou artefato sonoro.

2.2.1 *Callback* de Áudio e Tamanho de *Buffer*

Drivers de áudio operam no modelo ***pull callback***: o hardware solicita periodicamente um bloco de N amostras e o motor de áudio tem até o prazo do próximo bloco para entregá-lo (FONS, 2010). A relação entre *buffer* e latência é direta:

$$L = \frac{N}{f_s} \quad (1)$$

onde L é a latência em segundos, N o tamanho do bloco em amostras e f_s a taxa de amostragem. A 44.100 Hz, $N = 256$ corresponde a $\approx 5,8$ ms. Reduzir N diminui a latência perceptível, mas multiplica a frequência das interrupções — e, com ela, a pressão sobre o escalonador (KIM; HAHN, 2003).

2.2.2 ASIO vs. WDM/DirectSound

O Windows expõe duas rotas de áudio ao aplicativo. Pelo **WDM/DirectSound**, o fluxo passa pelo *Kernel Audio Mixer* (KMixer), responsável por misturar áudio de múltiplos aplicativos antes de enviá-lo ao hardware (Microsoft Corporation, 2023a). Cada ciclo de *buffer* acorda o KMixer, acrescentando latência e interrupções extras. Já o protocolo **ASIO** (*Audio Stream Input/Output*) entrega ao driver do fabricante acesso direto ao hardware, ignorando o KMixer (Microsoft Corporation, 2023a). O ganho é uma redução substancial de latência e de carga sobre o escalonador.

2.2.3 Por que DAWs não usam Escalonamento de Prazos

Liu e Layland (1973) formalizaram o escalonamento em tempo real com o algoritmo *Rate Monotonic*, e Buttazzo (2011) mostraram que a garantia formal de prazos exige análise de pior caso (WCET). DAWs, como sistemas *soft real-time*, dispensam esse rigor: limitam-se a elevar a prioridade das *threads* de áudio no escalonador genérico do SO, trocando garantia formal por simplicidade de implementação.

2.3 Gerenciamento de Memória e Paginação

Cada processo recebe do SO um espaço de endereçamento virtual contíguo e isolado, independente da memória física disponível (SILBERSCHATZ; GALVIN; GAGNE,

2015). A tradução entre endereço virtual e físico é feita pela MMU com base nas tabelas de páginas do *kernel*. Na **paginação por demanda**, uma página só é mapeada a um quadro físico quando acessada pela primeira vez — evento chamado de **falta de página** (*page fault*) (TANENBAUM; BOS, 2016). Resolvê-la exige suspender a *thread*, localizar (ou ler do disco) a página e retomar a execução: uma latência não determinística que pode variar de microssegundos a milissegundos.

Para áudio em tempo real, uma falta de página durante o processamento de um bloco é especialmente cara: a *thread* fica bloqueada enquanto o prazo do *buffer* avança (KIM; HAHN, 2003). Sistemas críticos fixam páginas na RAM via `mlock()` (Linux) ou `VirtualLock()` (Windows); DAWs comerciais costumam optar por alocação antecipada (*eager allocation*) como equivalente prático.

2.3.1 *Eager vs. Lazy Allocation*

Tanenbaum e Bos (2016) contrastam dois modelos de alocação:

- ***Eager allocation***: reserva e inicializa toda a memória prevista logo na inicialização do processo. Elimina faltas de página no caminho crítico, ao custo de uma pegada elevada em repouso.
- ***Lazy allocation***: aloca recursos só quando demandados. Mantém pegada mínima em repouso, mas introduz latência não determinística no momento da primeira alocação.

2.3.2 *Private Bytes*

O contador *Private Bytes* do Windows mede a memória virtual comprometida exclusivamente pelo processo — ignorando DLLs e demais regiões compartilhadas (STALLINGS, 2018). É a métrica mais fiel ao custo real de memória da aplicação, por refletir o que precisaria ser paginado caso o sistema ficasse sem RAM física. Em DAWs, o crescimento desse contador ao carregar trilhas reflete sobretudo os *buffers* de pré-leitura e as estruturas de decodificação mantidas em memória para reprodução de baixa latência.

3 METODOLOGIA

O presente estudo adota uma abordagem quantitativa e comparativa para analisar o gerenciamento de recursos e o desempenho de *Digital Audio Workstations* (DAWs) sob a ótica de Sistemas Operacionais. O foco reside na interação entre o *kernel* do Windows e os motores de áudio do **Ableton Live 12** e **FL Studio 24**.

3.1 Ambiente de Teste e Isolamento de Variáveis

Para garantir a reprodutibilidade, o ambiente foi configurado para minimizar a interferência de processos em segundo plano. Os testes foram executados em um notebook **ASUS Vivobook K3605ZF** com as seguintes especificações:

- **Processador:** Intel Core i5-12500H (12^a geração) — 12 núcleos (4 núcleos de desempenho + 8 núcleos de eficiência), 16 *threads*, frequência base de 3,1 GHz e *boost* de até 4,5 GHz.
- **Memória RAM:** 16 GB DDR4.
- **GPU:** NVIDIA (driver `nvlddmkm.sys` v581.29), relevante para a análise de DPCs do Teste 4.
- **Sistema Operacional:** Windows 11 (build 26200, x64).
- **Camada de Abstração de Áudio:** Cada DAW foi testada com seu driver de áudio nativo. O FL Studio utilizou o *FL Studio ASIO*; o Ableton Live utilizou seu driver nativo padrão (DirectSound/MME).
- **Configuração de saída:** O *Fruity Limiter* nativo do FL Studio foi removido do canal Master antes dos testes, garantindo paridade com a arquitetura aditiva do Ableton Live.
- **Antivírus:** O Windows Defender permaneceu ativo em todos os testes, em condições idênticas para ambas as DAWs.

3.2 Instrumentação e Coleta de Métricas

- **Performance Monitor (PerfMon):** Ferramenta nativa do Windows utilizada para capturar contadores do *kernel* a 1 Hz, exportados via `relog` para formato `.csv` (Microsoft Corporation, 2023b). Métricas coletadas:
 - % Tempo de Processador (`_Total`)

- Bytes Particulares (Private Bytes) por processo
- Contagem de Threads por processo
- **Process Explorer (Sysinternals):** Inspeção detalhada de *threads* individuais, prioridades de escalonamento e consumo de CPU por *thread*.
- **LatencyMon v7.31 (Resplendence):** Medição de latência de rotinas DPC e ISR, latência *interrupt-to-process* e contagem de *hard pagefaults* em nível de sistema.

3.3 Protocolo de Teste 1: Consumo em Repouso (*Idle*)

DAW aberta sem projeto carregado. Coleta de 2,5 minutos a 1 Hz via PerfMon.

3.4 Protocolo de Teste 2: Carga *I/O-bound* com 20 Trilhas AIFF

Reprodução simultânea em *loop* de 20 trilhas de áudio no formato **AIFF** (PCM não comprimido), evitando ciclos de CPU destinados à descompressão. Coleta de 3 minutos a 1 Hz.

3.5 Protocolo de Teste 3: Escalonamento de *Threads*

Inspeção via *Process Explorer* com o projeto de 20 trilhas em reprodução. Registraram-se: total de *threads* ativas, prioridade base e dinâmica da *thread* principal, módulos de origem das *threads* e CPU consumida por *thread*.

3.6 Protocolo de Teste 4: Latência com Driver Nativo (LatencyMon)

Medição da latência de driver com o **LatencyMon v7.31**, executado durante 2 minutos e 50 segundos com o projeto de 20 trilhas AIFF em reprodução em cada DAW, utilizando os drivers nativos sem alteração de configurações de *buffer*. O Windows Defender permaneceu ativo em ambos os testes, garantindo paridade de condições. Métricas coletadas: latência máxima e média *interrupt-to-process*, tempo máximo de rotinas DPC e ISR, percentual de CPU em DPCs/ISRs e contagem total de *hard pagefaults*.

3.7 Protocolo de Teste 5: Estresse CPU-bound e Ponto de Ruptura

Introdução de carga de processamento intensivo via 5 instâncias do plugin de reverb nativo de cada DAW inseridas no canal Master, com configuração padrão de fábrica. Com as 20 trilhas AIFF em reprodução simultânea e os reverbs ativos, o *buffer size* foi reduzido progressivamente (4096 → 2048 → 1024 → 512 → 256 → 128 *samples*), aguardando 20 segundos em cada etapa. O ponto de ruptura (*buffer underrun*) foi identificado pela

ocorrência de *clicks*, *pops* ou cortes no sinal de áudio. O FL Studio utilizou o driver **FL Studio ASIO**; o Ableton Live utilizou **MME/DirectX**, preservando as escolhas nativas de cada fabricante.

4 RESULTADOS

4.1 Teste 1 — Consumo em Repouso (*Idle*)

O Teste 1 estabeleceu o consumo de recursos de ambas as DAWs sem nenhum projeto carregado. Os dados foram coletados durante aproximadamente 2,5 minutos, resultando em 146 amostras válidas para o Ableton Live 12 e 145 para o FL Studio 24.

Tabela 1 – Resumo estatístico — Teste 1: consumo em repouso

Métrica	DAW	Mínimo	Média	Máximo
% CPU	Ableton Live 12	0,00%	10,95%	33,73%
	FL Studio (FL64)	0,00%	12,38%	37,72%
Private Bytes (MB)	Ableton Live 12	338,6	536,8	538,3
	FL Studio (FL64)	95,6	95,8	95,8
Threads ativas	Ableton Live 12	8	54,8	56
	FL Studio (FL64)	44	44,0	45

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.1.1 CPU em repouso

O FL Studio registrou média de 12,38%, ligeiramente superior ao Ableton Live (10,95%), com pico máximo de 37,72% frente a 33,73%. A diferença é sistemática ao longo de toda a sessão (Figura 1), evidenciando processos internos contínuos mesmo sem projeto ativo.

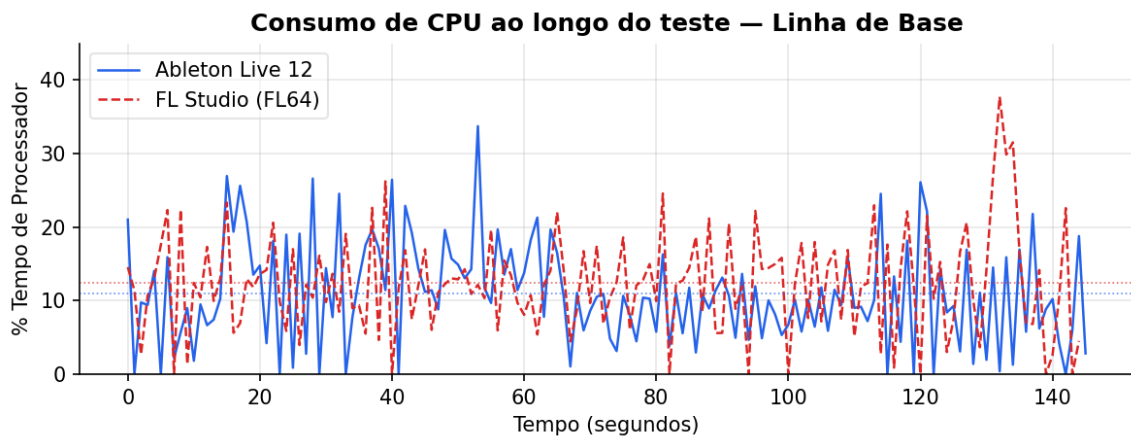


Figura 1 – Consumo de CPU em repouso — Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.1.2 RAM em repouso

O Ableton Live alocou em média 536,8 MB em repouso, valor $5,6\times$ superior ao FL Studio (95,8 MB), que manteve consumo praticamente constante (variação $< 0,2$ MB) (Figura 2).

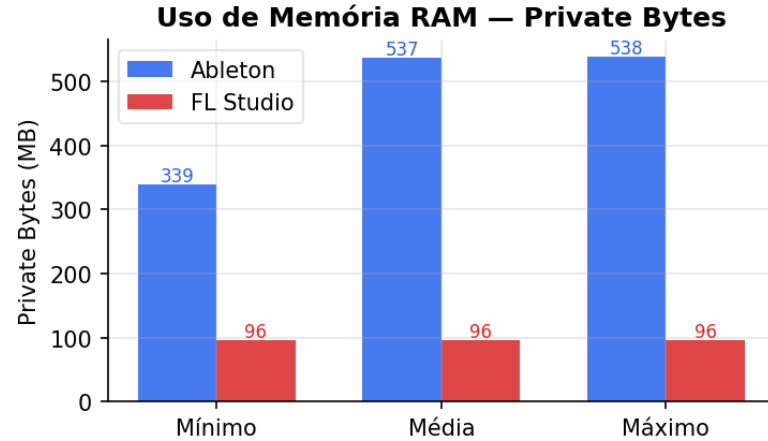


Figura 2 – Uso de RAM (*Private Bytes*) em repouso — Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.1.3 *Threads* em repouso

O Ableton Live manteve em média 54,8 *threads* ativas em repouso, contra 44 *threads* estáveis do FL Studio (Figura 3).

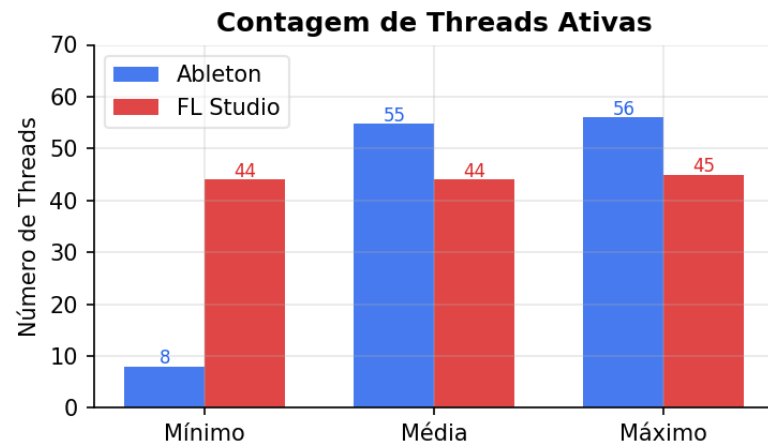


Figura 3 – Contagem de *threads* em repouso — Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2 Teste 2 — Carga *I/O-bound*: 20 Trilhas AIFF

O Teste 2 introduziu carga de trabalho real com a reprodução simultânea em *loop* de 20 trilhas de áudio AIFF (PCM não comprimido). Foram coletadas 178 amostras para

o Ableton Live 12 e 169 para o FL Studio 24, durante aproximadamente 3 minutos cada.

Tabela 2 – Resumo estatístico — Teste 2: 20 trilhas AIFF em reprodução

Métrica	DAW	Mínimo	Média	Máximo
% CPU	Ableton Live 12	0,00%	3,74%	20,11%
	FL Studio (FL64)	0,00%	4,80%	31,16%
Private Bytes (MB)	Ableton Live 12	601,6	601,7	604,5
	FL Studio (FL64)	137,6	147,6	147,6
Threads ativas	Ableton Live 12	54	55,3	56
	FL Studio (FL64)	26	27,0	28

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2.1 CPU com 20 trilhas

Com a carga *I/O-bound* ativa, ambas as DAWs apresentaram redução expressiva no consumo médio de CPU em relação ao repouso: o Ableton Live caiu de 10,95% para 3,74% (−6,21 pp) e o FL Studio de 12,38% para 4,80% (−7,58 pp). A Figura 4 ilustra o comportamento ao longo do teste, e a Figura 5 compara os dois cenários.

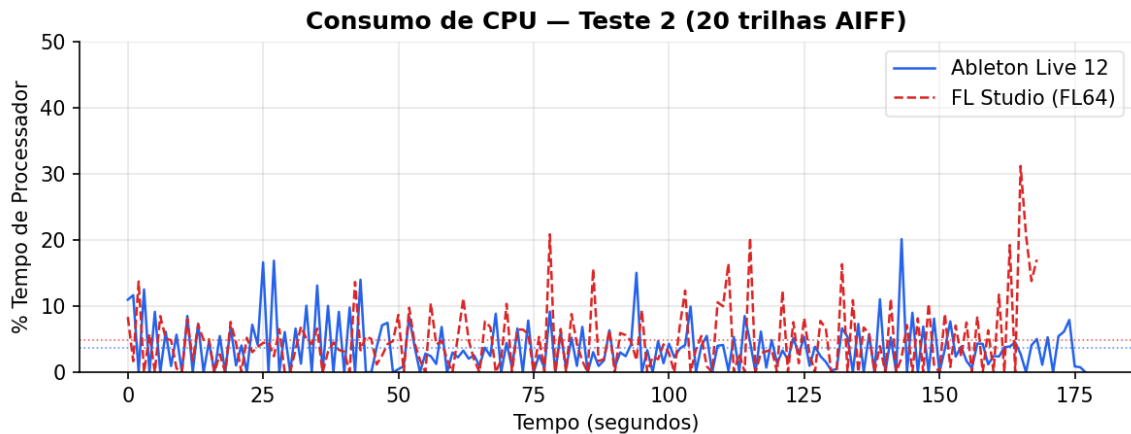


Figura 4 – Consumo de CPU durante reprodução de 20 trilhas AIFF — Teste 2.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2.2 RAM com 20 trilhas

O Ableton Live elevou seu consumo de 536,8 MB para 601,7 MB (+64,9 MB). O FL Studio partiu de 95,8 MB e estabilizou em 147,6 MB (+51,8 MB). O Ableton consumiu $4,1\times$ mais RAM que o FL Studio com 20 trilhas ativas (Figura 6).

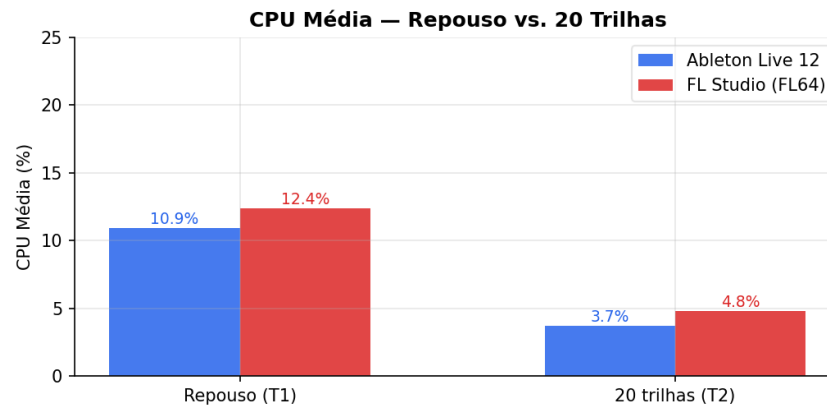


Figura 5 – CPU média comparativa: repouso (Teste 1) vs. 20 trilhas (Teste 2).

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

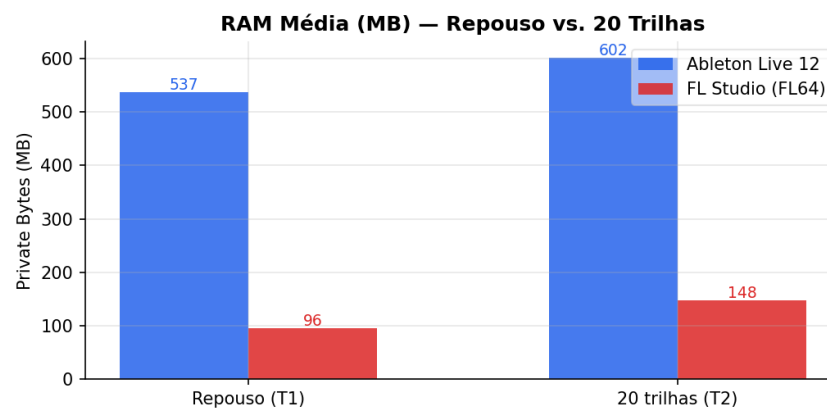


Figura 6 – RAM média comparativa: repouso (Teste 1) vs. 20 trilhas (Teste 2).

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2.3 *Threads* com 20 trilhas

O FL Studio reduziu sua contagem de 44 para 27 *threads* ao carregar o projeto (−38%), enquanto o Ableton Live manteve-se estável (54,8 → 55,3) (Figura 7).

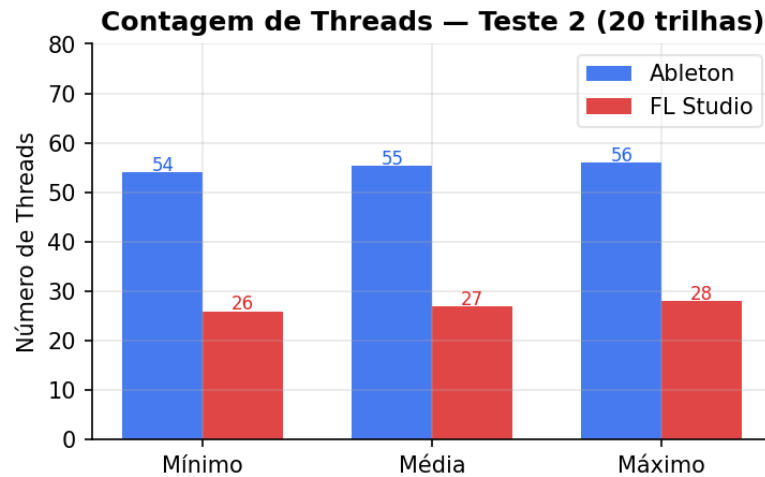


Figura 7 – Contagem de *threads* com 20 trilhas ativas — Teste 2.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.3 Teste 3 — Escalonamento de *Threads* (Process Explorer)

O Teste 3 inspecionou a composição interna de *threads* de cada DAW com o projeto de 20 trilhas AIFF em reprodução ativa via *Process Explorer*. A Tabela 3 resume os dados coletados.

Tabela 3 – Escalonamento de *threads* — Teste 3 (Process Explorer)

Parâmetro	FL Studio (FL64)	Ableton Live 12
Total de <i>threads</i>	29	55
Prioridade base (thread principal)	10 (Above Normal)	8 (Normal)
Prioridade dinâmica	12 (High)	10 (Above Normal)
CPU da thread principal	0,93%	0,75%
Thread <i>Time Critical</i> (15)?	Não	Não
Módulo dominante	QuickFontCache	ucrtbase
Thread de driver de áudio	ilwasapi2asio_x64	DSOUND.dll

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

4.3.1 Prioridade de escalonamento

O FL Studio agenda sua *thread* principal com prioridade base 10 (*Above Normal*) e dinâmica 12 (*High*), enquanto o Ableton Live utiliza base 8 (*Normal*) e dinâmica 10. Nenhuma das duas DAWs atingiu o nível *Time Critical* (15) (Figura 8).

4.3.2 Composição de *threads* por módulo

No FL Studio, 14 das 29 *threads* pertencem ao QuickFontCache (renderização de fontes da UI) e apenas 5 ao FLEngine_x64 (motor de áudio). No Ableton Live, ≈ 30 das

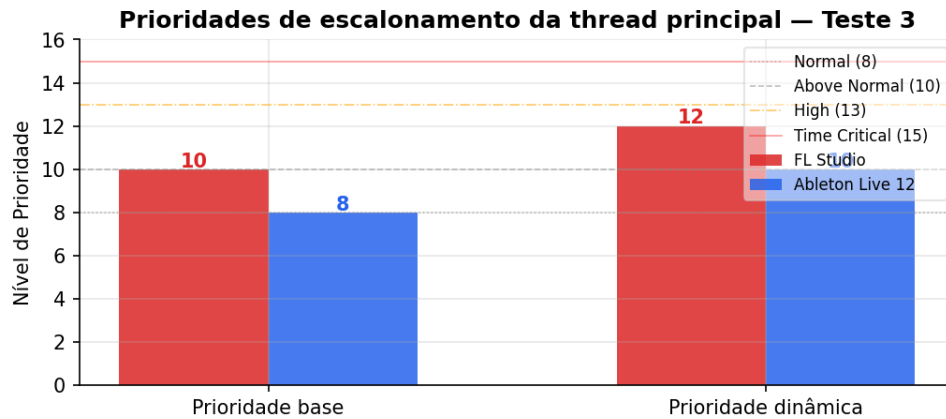


Figura 8 – Prioridades de escalonamento da *thread* principal — Teste 3.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

55 *threads* provêm do `ucrtbase.dll` (*thread pool* do C runtime), com o driver de áudio representado por `DSOUND.dll` (Figura 9).

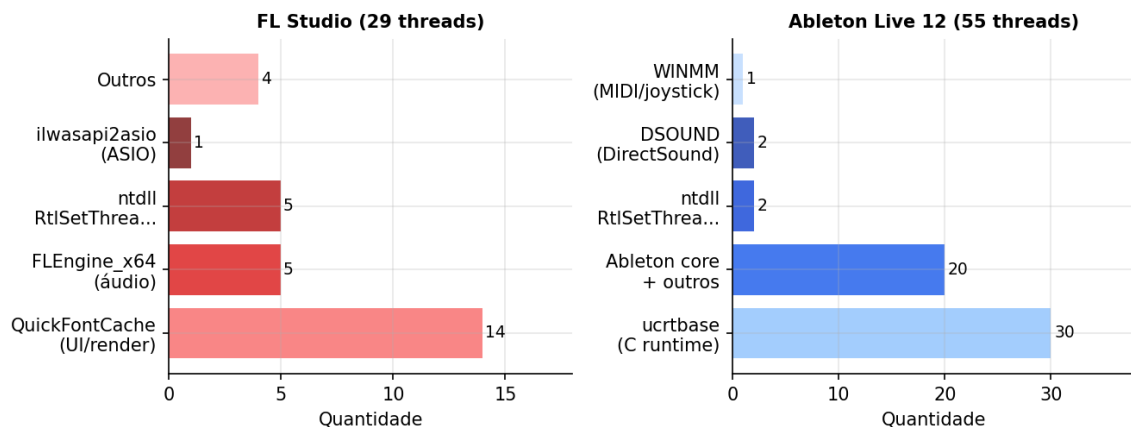


Figura 9 – Composição de *threads* por módulo — Teste 3.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

4.4 Teste 4 — Latência com Driver Nativo (LatencyMon)

O Teste 4 mediu a latência de *interrupt-to-process* e o comportamento de rotinas DPC e ISR durante a reprodução das 20 trilhas AIFF, com duração de 2 minutos e 50 segundos por DAW. A Tabela 4 apresenta o resumo.

4.4.1 Latência *interrupt-to-process*

O Ableton Live registrou pico de $805,40\ \mu\text{s}$ — dentro do limiar aceitável para áudio em tempo real. O FL Studio atingiu $5274,90\ \mu\text{s}$ ($6,5\times$ maior), levando o LatencyMon a classificar o sistema como *não apto*, com risco de *buffer underruns* (Figura 10). O driver responsável pelo maior DPC no FL Studio foi o `nvlddmkm.sys` (NVIDIA, $551,84\ \mu\text{s}$), e o

Tabela 4 – Resultados de latência — Teste 4 (LatencyMon v7.31)

Métrica	Ableton Live 12	FL Studio (FL64)
Conclusão do sistema	Apto	Não apto
Latência máx. int.-to-proc. (μs)	805,40	5274,90
Latência méd. int.-to-proc. (μs)	8,50	14,68
DPC máximo (μs)	449,81	551,84
ISR máximo (μs)	267,52	267,35
Tempo total em DPCs (%)	0,074	0,111
Tempo total em ISRs (%)	0,023	0,045
<i>Hard pagefaults</i> totais	306	380
Driver com maior DPC	tcpip.sys	nvlddmkm.sys
Driver com maior ISR	Wdf01000.sys	

Fonte: dados coletados pelos autores via LatencyMon v7.31, 26 maio 2026.

processo com maior contagem de *hard pagefaults* foi o `mmpeng.exe` (Windows Defender) em ambos os testes.

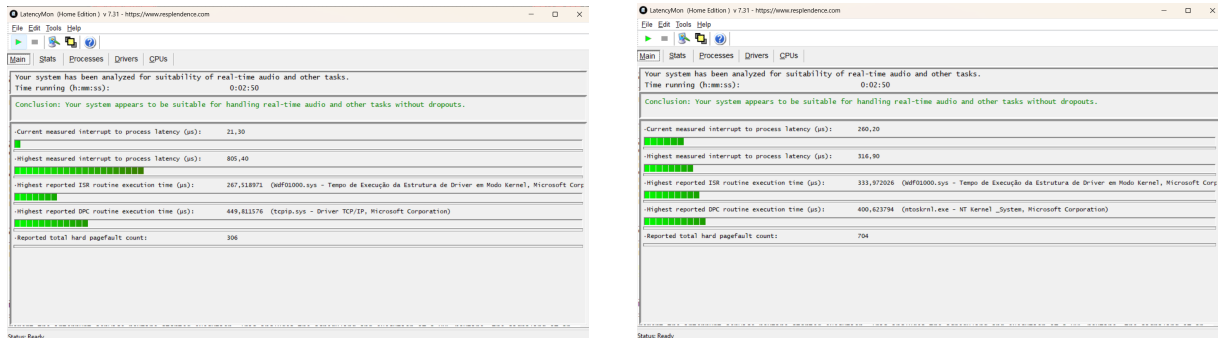


Figura 10 – Tela principal do LatencyMon — Ableton Live 12 (esquerda, conclusão verde: apto) e FL Studio (direita, conclusão vermelha: não apto).

Latência máxima *interrupt-to-process*: 805 μs vs. 5274 μs .

Fonte: dados coletados pelos autores via LatencyMon v7.31, 26 maio 2026.

4.4.2 *Hard pagefaults* e tempo de CPU em DPCs/ISRs

O FL Studio registrou maior contagem de *hard pagefaults* (380 vs. 306) e maior percentual de CPU em DPCs (0,111% vs. 0,074%) e ISRs (0,045% vs. 0,023%) (Figura 11).

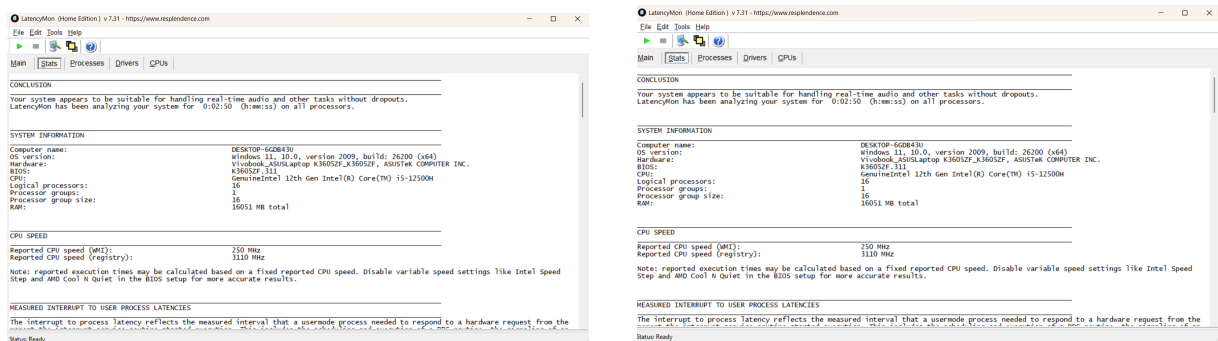


Figura 11 – Relatório detalhado do LatencyMon — Ableton Live 12 (esquerda) e FL Studio (direita). Detalhe para os *hard pagefaults* (306 vs. 380) e o driver *nvlldmkm.sys* (NVIDIA) como principal responsável pelo DPC elevado no FL Studio.

Fonte: dados coletados pelos autores via LatencyMon v7.31, 26 maio 2026.

4.5 Teste 5 — Estresse CPU-bound e Ponto de Ruptura

O Teste 5 introduziu carga de processamento de ponto flutuante por meio de 5 instâncias do reverb nativo de cada DAW no canal Master, combinadas à reprodução das 20 trilhas AIFF. O *buffer size* foi reduzido progressivamente até o surgimento de *dropouts* audíveis. A Tabela 5 sintetiza os resultados.

Tabela 5 – Ponto de ruptura — Teste 5 (estresse CPU-bound)

Buffer (smp)	Latência (ms)	FL Studio (ASIO)	Ableton (MME/DirectX)
4096	92,9	—	Estável
2048	46,4	—	Estável
1024	23,2	—	Glitch
512	11,6	Estável	Glitch piora
384	9,0	Estável	—
256	5,8	Mínimo estável	Glitch forte
128	2,9	Glitch	—

Fonte: dados coletados pelos autores, 27 maio 2026.

4.5.1 Ponto de ruptura por driver

O FL Studio, operando com o driver **FL Studio ASIO**, manteve-se estável até 256 *samples* (6 ms), apresentando *dropout* apenas ao atingir 128 *samples*. O Ableton Live, utilizando **MME/DirectX**, iniciou os cortes já em 1024 *samples* (23,2 ms), com degradação progressiva até 256 *samples*. A Figura 12 ilustra a relação entre *buffer size* e latência ao longo do teste, com os pontos de ruptura assinalados.

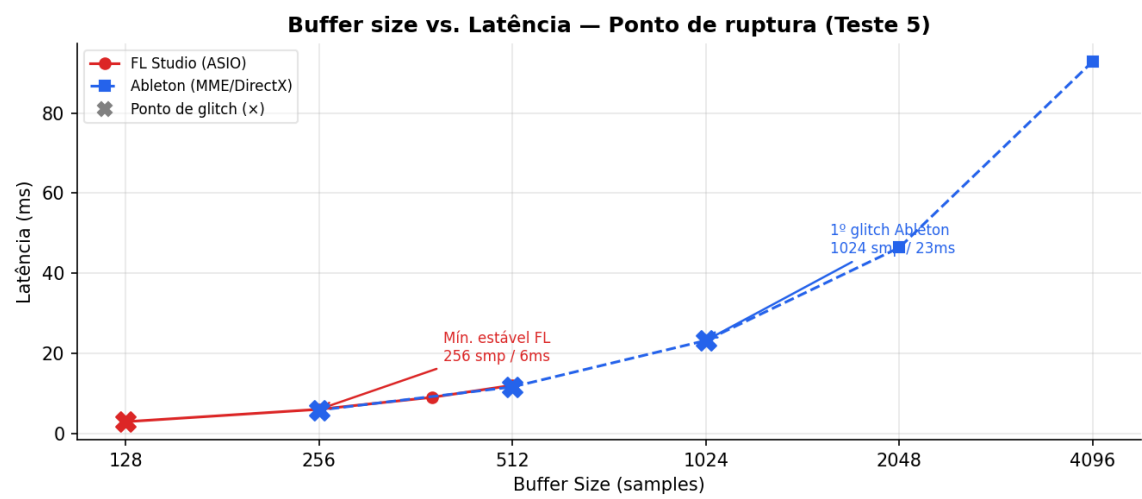


Figura 12 – Buffer size vs. latência ao longo do Teste 5. O marcador × indica o ponto de ocorrência de *dropout* audível em cada DAW.
Fonte: dados coletados pelos autores, 27 maio 2026.

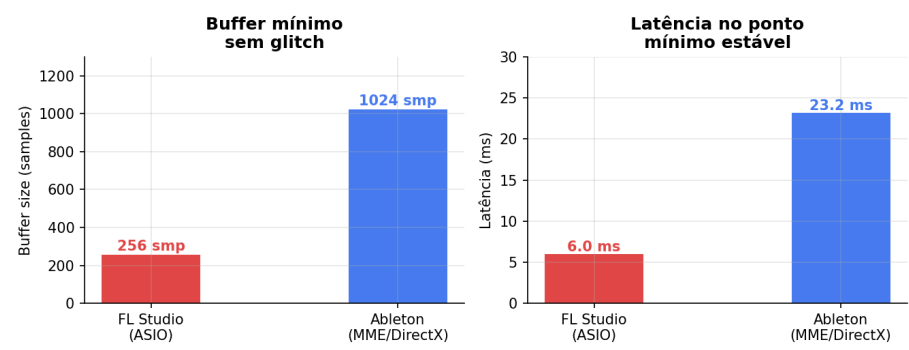


Figura 13 – Comparativo do *buffer* mínimo estável e latência correspondente — Teste 5.
Fonte: dados coletados pelos autores, 27 maio 2026.

4.6 Consumo de CPU e Comportamento Multithread

4.7 Análise de Latência e Buffer

5 DISCUSSÃO

5.1 Análise Comparativa — Teste 1 (Repouso)

Os resultados do Teste 1 revelam perfis de inicialização distintos. O **Ableton Live** adota *eager allocation*: aloca memória e cria *threads* antecipadamente (≈ 537 MB, 55 *threads*), garantindo baixa latência de despacho desde a inicialização (SILBERSCHATZ; GALVIN; GAGNE, 2015). O **FL Studio** demonstrou *lazy allocation*: mantém apenas estruturas mínimas em repouso (≈ 96 MB, 44 *threads*), postergando alocações para o carregamento do projeto (TANENBAUM; BOS, 2016).

5.2 Análise Comparativa — Teste 2 (20 Trilhas AIFF)

O Teste 2 revelou três achados relevantes. Primeiro, a **redução de CPU sob carga *I/O-bound*** em ambas as DAWs é explicada pelo driver de áudio nativo: ao assumir controle exclusivo do hardware, elimina a sobrecarga do *Kernel Mixer* do Windows. Segundo, o **crescimento proporcional de RAM** foi semelhante em termos absolutos (+65 MB no Ableton, +52 MB no FL Studio), indicando custo por trilha AIFF comparável entre as arquiteturas. Terceiro, a **queda de *threads* no FL Studio** (de 44 para 27) indica consolidação de *threads* genéricas em especializadas de áudio ao carregar o projeto (STALLINGS, 2017).

5.3 Análise Comparativa — Teste 3 (Escalonamento de *Threads*)

O **FL Studio** opera com prioridade mais elevada (base 10, dinâmica 12), obtendo maior fatia de CPU em situações de concorrência (SILBERSCHATZ; GALVIN; GAGNE, 2015). Contudo, 14 das 29 *threads* pertencem ao **QuickFontCache**, indicando que parte do orçamento de CPU é consumida pela interface gráfica, e não pelo motor de áudio. O **Ableton Live**, com prioridade base inferior (8), distribui sua carga em 55 *threads* via *thread pool* do **ucrtbase**, favorecendo escalabilidade nos 16 *threads* lógicos do i5-12500H (TANENBAUM; BOS, 2016). Nenhuma das DAWs usa prioridade *Time Critical* (15), confirmando o enquadramento *soft real-time* (STALLINGS, 2017).

5.4 Análise Comparativa — Teste 4 (Latência)

O Teste 4 revelou a diferença mais crítica entre as duas DAWs. O Ableton Live manteve latência máxima dentro do limiar aceitável ($805\ \mu\text{s}$), enquanto o FL Studio atingiu $5274\ \mu\text{s}$ — classificado como problemático pelo LatencyMon. O driver **nvlddmkm.sys**

(NVIDIA) foi o principal responsável pelos DPCs elevados durante o teste do FL Studio, e o `msmpeng.exe` (Windows Defender) gerou o maior número de *hard pagefaults* em ambos os casos (SILBERSCHATZ; GALVIN; GAGNE, 2015). O fato de o Ableton ter permanecido abaixo do limiar crítico nas mesmas condições sugere que sua arquitetura de *eager allocation* — com memória pré-alocada e mais *threads* prontas — reduz a frequência de *hard pagefaults* e, consequentemente, a latência máxima de resposta (TANENBAUM; BOS, 2016).

5.5 Análise Comparativa — Teste 5 (Estresse CPU-bound)

O resultado do Teste 5 deve ser interpretado com cautela, pois as duas DAWs utilizaram drivers de áudio distintos — diferença que constitui, em si mesma, um dado relevante sobre as escolhas arquiteturais de cada fabricante.

O **FL Studio** com driver **ASIO** sustentou reprodução estável até 256 *samples* (6 ms), confirmando o benefício do acesso direto ao hardware proporcionado pelo protocolo ASIO: ao contornar o *Kernel Audio Mixer* (KMixer) do Windows, o driver reduz o número de interrupções por ciclo de *buffer* e diminui a pressão sobre o escalonador (Microsoft Corporation, 2023a). Esse resultado é consistente com a prioridade de *thread* mais elevada (base 10, dinâmica 12) observada no Teste 3, que garante ao motor de áudio maior janela de tempo para preencher cada bloco antes do prazo.

O **Ableton Live** com **MME/DirectX** iniciou os *dropouts* em 1024 *samples* (23,2 ms) — quatro vezes acima do ponto de ruptura do FL Studio. Esse comportamento é previsível à luz do referencial teórico: o caminho WDM/DirectX passa pelo KMixer, que adiciona latência e interrupções extras a cada ciclo de *buffer* (Microsoft Corporation, 2023a; KIM; HAHN, 2003). Sob carga CPU-bound (5 reverbs em convolução simultâneos), esse custo adicional torna inviável a manutenção de buffers menores sem *underruns*.

Ressalta-se que a comparação direta dos pontos de ruptura reflete sobretudo a diferença entre os modelos de driver, e não necessariamente a capacidade intrínseca do motor de áudio de cada DAW. Em um cenário com ambas as DAWs utilizando ASIO, o resultado poderia ser diferente — hipótese que constitui uma limitação deste estudo e uma direção para trabalhos futuros.

6 CONCLUSÃO

O presente estudo demonstrou que, sob a perspectiva do gerenciamento de recursos do Sistema Operacional, Ableton Live e FL Studio seguem abordagens arquiteturais bem diferentes. Em resumo, os resultados obtidos mostram que o Ableton Live prioriza a alocação antecipada (*eager allocation*): a aplicação retém antecipadamente o espaço necessário e estabelece um extenso *pool* de *threads* desde o início para garantir baixa latência de despacho e capacidade de dividir o trabalho de forma eficiente entre os 16 *threads* lógicos do hardware utilizado. Em contrapartida, o FL Studio baseia-se em alocação sob demanda (*lazy allocation*), operando com estruturas mínimas até que a carga de trabalho seja exigida.

Verificou-se também que o FL Studio tenta contornar gargalos ao atribuir prioridade de escalonamento mais alta (nível dinâmico 12) à sua *thread* principal. Contudo, a análise revelou que parte significativa desse esforço computacional não é utilizada pelo motor de áudio, mas sim consumida pela renderização da interface gráfica (módulo `QuickFontCache`). O teste de latência confirmou essa limitação: sob as mesmas condições de sistema — incluindo o driver NVIDIA (`nvlddmkm.sys`) e o Windows Defender (`msmpeng.exe`) ativos — o Ableton Live manteve-se dentro do limiar aceitável ($805\ \mu\text{s}$), enquanto o FL Studio atingiu $5274\ \mu\text{s}$, sendo classificado pelo LatencyMon como *não apto* para áudio em tempo real sem *dropouts*.

Por fim, constatou-se que o uso de *drivers* nativos mitiga a sobrecarga de CPU ao eliminar a intervenção do *Kernel Mixer* do Windows, e que ambas as DAWs operam estritamente como sistemas de tempo real não crítico (*soft real-time*), sem atingir níveis de prioridade *Time Critical*.

No que diz respeito às aplicações práticas, os resultados fornecem diretrizes para dimensionamento de hardware e design de software. Para usuários do Ableton Live, estações de trabalho com ampla RAM e processadores com alto número de núcleos são mais vantajosas. Para o ecossistema do FL Studio, processadores com alto desempenho *single-core* podem compensar a concentração de carga na *thread* principal. Do ponto de vista da Engenharia de Software, os dados evidenciam a necessidade de isolar de forma mais eficiente as *threads* de processamento de áudio das rotinas de interface gráfica, garantindo que o escalonador do SO não penalize a entrega de dados sensíveis ao tempo em favor da renderização visual.

REFERÊNCIAS

BUTTAZZO, G. C. **Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications**. 3. ed. New York: Springer, 2011.

FONS, J. Real-time audio processing on general purpose operating systems. In: **Linux Audio Conference**. Utrecht: [s.n.], 2010.

KIM, Y.; HAHN, M. Improving real-time audio performance under general purpose operating systems. **IEICE Transactions on Information and Systems**, E86-D, n. 1, p. 192–196, 2003.

LIU, C. L.; LAYLAND, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. **Journal of the ACM**, v. 20, n. 1, p. 46–61, 1973.

Microsoft Corporation. **Audio Drivers for Windows: ASIO Architecture**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-hardware/drivers/audio>>. Acesso em: 26 maio 2026.

Microsoft Corporation. **Windows Performance Monitor: Technical Reference**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/perfmon>>. Acesso em: 26 maio 2026.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. Tradução de: Operating System Concepts, 9th ed.

STALLINGS, W. **Operating Systems: Internals and Design Principles**. 9. ed. Hoboken: Pearson, 2017.

STALLINGS, W. **Arquitetura e Organização de Computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2018. Tradução de: Computer Organization and Architecture, 10th ed.

TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Tradução de: Modern Operating Systems, 4th ed.